PYTHON COMPLETO

Guia Técnico para o LIBERTUS

Do básico ao avançado — automação, segurança, APIs, IA

1. FUNDAMENTOS

Tipos de Dados

Python tem tipos nativos poderosos: - int, float, complex — números - str — texto imutável - bool — True / False - list — lista mutável ordenada - tuple — lista imutável - dict — chave:valor (hash map) - set — conjunto sem duplicatas - bytes — dados binários

```
x = 42
pi = 3.14
nome = "LIBERTUS"
ativo = True
lista = [1, 2, 3, "quatro"]
tupla = (10, 20, 30)
mapa = {"chave": "valor", "id": 1}
conjunto = {1, 2, 3, 2, 1}  # {1, 2, 3}
binario = b"dados"

# Verificar tipo
print(type(x))       # <class 'int'>
print(isinstance(x, int))  # True
```

Strings — Operações Essenciais

Strings são imutáveis. Toda operação retorna nova string.

```
s = "LIBERTUS é poderoso"

# Básico
print(s.upper())           # LIBERTUS É PODEROSO
print(s.lower())           # libertus é poderoso
print(s.replace("é", "=")) # LIBERTUS = poderoso
print(s.split(" "))        # ['LIBERTUS', 'é', 'poderoso']
print(len(s))              # 19
print(s[0:8])              # LIBERTUS
print(s.strip())           # remove espaços nas bordas

# F-strings (use sempre isso)
nome = "Lorran"
versao = 4.0
msg = f"Criado por {nome} — versão {versao:.1f}"
print(msg)  # Criado por Lorran — versão 4.0

# Multiline
texto = """
Linha 1
Linha 2
Linha 3
"""

# Verificações
print("LIBER" in s)        # True
```

```
print(s.startswith("LIB")) # True
print(s.endswith("oso"))   # True
print(s.find("é"))         # 9

# Join — eficiente para concatenar listas
palavras = ["a", "b", "c"]
print("-".join(palavras))  # a-b-c
```

Listas e Comprehensions

List comprehensions são uma das features mais poderosas do Python. Substituem loops for + append em uma linha.

```
# Lista básica
nums = [1, 2, 3, 4, 5]
nums.append(6)       # adiciona no fim
nums.insert(0, 0)    # insere na posição
nums.pop()           # remove e retorna último
nums.remove(3)       # remove primeira ocorrência
nums.sort()          # ordena in-place
nums.reverse()       # inverte in-place

# Slicing
print(nums[1:4])     # índices 1,2,3
print(nums[::-1])    # invertida
print(nums[::2])     # de 2 em 2

# List comprehension
quadrados = [x**2 for x in range(10)]
pares = [x for x in range(20) if x % 2 == 0]
flat = [item for sub in [[1,2],[3,4]] for item in sub]

# Dict comprehension
mapa = {k: v for k, v in zip("abc", [1,2,3])}
# {'a': 1, 'b': 2, 'c': 3}

# Set comprehension
unicos = {x % 3 for x in range(10)}
```

Dicionários

Dicionários são o coração do Python — usados em JSON, APIs, configs, memória.

```
d = {"nome": "LIBERTUS", "versao": 4, "ativo": True}

# Acesso seguro
nome = d.get("nome", "desconhecido")
versao = d.get("versao", 0)

# Operações
d["novo"] = "valor"          # adiciona/atualiza
del d["ativo"]               # remove
chaves = list(d.keys())
```

```
valores = list(d.values())
pares = list(d.items())      # lista de (chave, valor)

# Iterar
for chave, valor in d.items():
    print(f"{chave}: {valor}")

# Merge (Python 3.9+)
a = {"x": 1}
b = {"y": 2}
c = a | b  # {"x": 1, "y": 2}

# Dict aninhado
config = {
    "db": {"host": "localhost", "port": 5432},
    "api": {"key": "abc", "timeout": 30}
}
host = config["db"]["host"]
timeout = config.get("api", {}).get("timeout", 60)
```

2. FUNÇÕES E PROGRAMAÇÃO FUNCIONAL

Funções — Tudo que Precisa Saber

Funções são cidadãos de primeira classe em Python — podem ser passadas como argumentos, retornadas, armazenadas em variáveis.

```
# Básico
def somar(a, b):
    return a + b

# Default args
def conectar(host, porta=5432, timeout=30):
    return f"{host}:{porta} (timeout={timeout}s)"

# *args — quantidade variável de argumentos posicionais
def soma_tudo(*args):
    return sum(args)

soma_tudo(1, 2, 3, 4, 5)  # 15

# **kwargs — quantidade variável de argumentos nomeados
def criar_usuario(**kwargs):
    return {k: v for k, v in kwargs.items()}

criar_usuario(nome="Lorran", role="admin", ativo=True)

# Type hints (use sempre — melhora autocomplete e legibilidade)
def processar(entrada: str, limite: int = 100) -> dict:
    return {"entrada": entrada[:limite], "tamanho": len(entrada)}

# Retornar múltiplos valores
def min_max(lista: list) -> tuple[int, int]:
    return min(lista), max(lista)

minimo, maximo = min_max([3, 1, 4, 1, 5, 9])
```

Lambda, Map, Filter, Reduce

Programação funcional torna o código mais conciso e expressivo.

```
import functools

# Lambda — função anônima de uma linha
dobrar = lambda x: x * 2
somar = lambda a, b: a + b

# Map — aplica função a cada elemento
numeros = [1, 2, 3, 4, 5]
dobrados = list(map(lambda x: x * 2, numeros))
# [2, 4, 6, 8, 10]
```

```
# Filter — filtra elementos
pares = list(filter(lambda x: x % 2 == 0, numeros))
# [2, 4]

# Reduce — acumula resultado
produto = functools.reduce(lambda a, b: a * b, numeros)
# 120

# Sorted com key
dados = [{"nome": "Carlos", "score": 8}, {"nome": "Ana", "score": 9}]
ordenado = sorted(dados, key=lambda x: x["score"], reverse=True)

# Combinando
resultado = list(map(
    lambda x: x**2,
    filter(lambda x: x % 2 == 0, range(10))
))
# [0, 4, 16, 36, 64]
```

Decoradores

Decoradores modificam o comportamento de funções sem alterar o código delas. Usados em frameworks web, logging, cache, autenticação.

```
import time
import functools

# Decorador de timing
def cronometrar(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        inicio = time.time()
        resultado = func(*args, **kwargs)
        fim = time.time()
        print(f"{func.__name__} levou {fim-inicio:.3f}s")
        return resultado
    return wrapper

# Decorador de retry
def retry(vezes=3, delay=1):
    def decorador(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for tentativa in range(vezes):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    if tentativa == vezes - 1:
                        raise
                    time.sleep(delay)
        return wrapper
    return decorador

# Uso
```

```
@cronometrar
@retry(vezes=3, delay=0.5)
def chamar_api(url: str) -> dict:
    import requests
    return requests.get(url).json()
```

3. CLASSES E ORIENTAÇÃO A OBJETOS

Classes Completas

Python usa OOP com herança, propriedades, métodos especiais (dunder methods).

```
from dataclasses import dataclass, field
from typing import Optional
from datetime import datetime

class Agente:
    # Atributo de classe (compartilhado por todas as instâncias)
    versao = "4.0"
    _instancias = 0

    def __init__(self, nome: str, modelo: str = "groq"):
        self.nome = nome
        self.modelo = modelo
        self._memoria = []          # atributo privado (convenção)
        self.__id = id(self)        # atributo muito privado (name mangling)
        self.criado_em = datetime.now()
        Agente._instancias += 1

    # Property — acesso controlado
    @property
    def memoria(self) -> list:
        return self._memoria.copy()  # retorna cópia, não referência

    @property
    def tamanho_memoria(self) -> int:
        return len(self._memoria)

    # Método de instância
    def memorizar(self, item: str) -> None:
        self._memoria.append({"item": item, "ts": datetime.now().isoformat()})

    # Método de classe
    @classmethod
    def total_instancias(cls) -> int:
        return cls._instancias

    # Método estático
    @staticmethod
    def validar_modelo(modelo: str) -> bool:
        return modelo in ["groq", "claude", "ollama"]

    # Dunder methods
    def __str__(self) -> str:
        return f"Agente({self.nome}, modelo={self.modelo})"

    def __repr__(self) -> str:
        return f"Agente(nome={self.nome!r}, modelo={self.modelo!r})"
```

```
    def __len__(self) -> int:
        return self.tamanho_memoria

    def __contains__(self, item: str) -> bool:
        return any(m["item"] == item for m in self._memoria)

# Dataclass — mais simples para structs
@dataclass
class Memoria:
    titulo: str
    conteudo: str
    peso: float = 5.0
    tags: list = field(default_factory=list)
    criada_em: str = field(default_factory=lambda: datetime.now().isoformat())

    def esta_viva(self) -> bool:
        return self.peso > 1.0
```

Herança e Polimorfismo

```
class LLMBase:
    def __init__(self, modelo: str, api_key: str):
        self.modelo = modelo
        self.api_key = api_key

    def responder(self, prompt: str) -> str:
        raise NotImplementedError("Subclasse deve implementar responder()")

    def esta_disponivel(self) -> bool:
        return bool(self.api_key)

class GroqClient(LLMBase):
    BASE_URL = "https://api.groq.com/openai/v1"

    def responder(self, prompt: str) -> str:
        # implementação específica do Groq
        import httpx
        r = httpx.post(
            f"{self.BASE_URL}/chat/completions",
            headers={"Authorization": f"Bearer {self.api_key}"},
            json={"model": self.modelo, "messages": [{"role": "user", "content": prompt}]}
        )
        return r.json()["choices"][0]["message"]["content"]

class ClaudeClient(LLMBase):
    def responder(self, prompt: str) -> str:
        import anthropic
        client = anthropic.Anthropic(api_key=self.api_key)
        msg = client.messages.create(
            model=self.modelo,
            max_tokens=1024,
            messages=[{"role": "user", "content": prompt}]
        )
        return msg.content[0].text
```

```python
# Polimorfismo — mesma interface, comportamentos diferentes
def processar_com_llm(llm: LLMBase, prompt: str) -> str:
    if not llm.esta_disponivel():
        return "LLM não disponível"
    return llm.responder(prompt)
```

4. TRATAMENTO DE ERROS E CONTEXTO

Exceções

```
# Hierarquia de exceções
try:
    resultado = 10 / 0
except ZeroDivisionError as e:
    print(f"Divisão por zero: {e}")
except (TypeError, ValueError) as e:
    print(f"Tipo ou valor inválido: {e}")
except Exception as e:
    print(f"Erro genérico: {e}")
    raise  # re-lança a exceção
else:
    print("Sem erros!")  # executa se não houve exceção
finally:
    print("Sempre executa")  # limpeza

# Criar exceções customizadas
class LibertusError(Exception):
    """Exceção base do LIBERTUS"""
    pass

class ConexaoError(LibertusError):
    def __init__(self, servico: str, detalhe: str):
        self.servico = servico
        self.detalhe = detalhe
        super().__init__(f"Falha ao conectar em {servico}: {detalhe}")

class ModeloIndisponivelError(LibertusError):
    pass

# Uso
try:
    raise ConexaoError("Supabase", "timeout após 30s")
except ConexaoError as e:
    print(f"Serviço: {e.servico}")
    print(f"Detalhe: {e.detalhe}")
```

Context Managers

```
# Context manager com class
class ConexaoDB:
    def __init__(self, url: str):
        self.url = url
        self.conn = None

    def __enter__(self):
        print(f"Conectando em {self.url}")
        self.conn = {"url": self.url, "ativo": True}
        return self.conn
```

```
    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Fechando conexão")
        if self.conn:
            self.conn["ativo"] = False
        return False  # não suprime exceções

# Context manager com contextlib
from contextlib import contextmanager

@contextmanager
def temporizador(nome: str):
    import time
    inicio = time.time()
    try:
        yield
    finally:
        fim = time.time()
        print(f"{nome}: {fim-inicio:.3f}s")

# Uso
with ConexaoDB("postgresql://localhost/libertus") as conn:
    print(f"Conectado: {conn}")

with temporizador("scan"):
    import time; time.sleep(0.1)
```

5. ARQUIVOS, JSON E AMBIENTE

Arquivos e JSON

```
import json
from pathlib import Path

# Pathlib — sempre use no lugar de os.path
base = Path("/home/libertus/LIBERTUS")
core = base / "core"
arquivo = core / "config.json"

# Criar diretórios
arquivo.parent.mkdir(parents=True, exist_ok=True)

# Ler e escrever texto
arquivo.write_text("conteúdo", encoding="utf-8")
conteudo = arquivo.read_text(encoding="utf-8")

# JSON
dados = {"nome": "LIBERTUS", "versao": 4, "ativo": True}

# Serializar
json_str = json.dumps(dados, indent=2, ensure_ascii=False)
arquivo.write_text(json_str, encoding="utf-8")

# Deserializar
dados_lidos = json.loads(arquivo.read_text(encoding="utf-8"))

# Listar arquivos
for py_file in core.glob("*.py"):
    print(py_file.name, py_file.stat().st_size, "bytes")

# Verificações
print(arquivo.exists())
print(arquivo.is_file())
print(arquivo.suffix)   # .json
print(arquivo.stem)     # config
```

Variáveis de Ambiente e Config

```
import os
from pathlib import Path

# Ler ambiente
api_key = os.environ.get("GROQ_API_KEY", "")
debug = os.environ.get("DEBUG", "false").lower() == "true"

# Carregar .env manualmente (sem depender de python-dotenv)
def carregar_env(caminho: str = ".env") -> dict:
    env = {}
    env_path = Path(caminho)
```

```
    if not env_path.exists():
        return env
    for linha in env_path.read_text().splitlines():
        linha = linha.strip()
        if not linha or linha.startswith("#"):
            continue
        if "=" in linha:
            chave, valor = linha.split("=", 1)
            env[chave.strip()] = valor.strip().strip('"').strip("'")
            os.environ[chave.strip()] = env[chave.strip()]
    return env

# Config com dataclass
from dataclasses import dataclass

@dataclass
class Config:
    groq_key: str = ""
    supabase_url: str = ""
    supabase_key: str = ""
    debug: bool = False
    modelo: str = "llama-3.3-70b-versatile"

    @classmethod
    def do_ambiente(cls) -> "Config":
        return cls(
            groq_key=os.environ.get("GROQ_API_KEY", ""),
            supabase_url=os.environ.get("SUPABASE_URL", ""),
            supabase_key=os.environ.get("SUPABASE_KEY", ""),
            debug=os.environ.get("DEBUG", "false") == "true",
        )
```

6. REQUESTS, APIS E HTTP

HTTP com httpx

httpx é o cliente HTTP moderno do Python — suporta sync e async. É o que o LIBERTUS usa internamente.

```
import httpx
import json

# GET simples
r = httpx.get("https://api.github.com/users/lorran562")
print(r.status_code)   # 200
print(r.json())        # dict com dados do usuário

# POST com JSON
payload = {"model": "llama-3.3-70b-versatile", "messages": [...]}
headers = {"Authorization": "Bearer gsk_...", "Content-Type": "application/json"}

r = httpx.post(
    "https://api.groq.com/openai/v1/chat/completions",
    headers=headers,
    json=payload,
    timeout=30
)

# Tratar resposta
if r.status_code == 200:
    dados = r.json()
    resposta = dados["choices"][0]["message"]["content"]
else:
    print(f"Erro {r.status_code}: {r.text}")

# Client reutilizável (mais eficiente — mantém conexão)
with httpx.Client(
    base_url="https://api.groq.com",
    headers={"Authorization": "Bearer gsk_..."},
    timeout=30
) as client:
    r = client.post("/openai/v1/chat/completions", json=payload)
    r = client.get("/openai/v1/models")

# Retry automático
from httpx import Client
import time

def get_com_retry(url: str, tentativas: int = 3) -> dict:
    for i in range(tentativas):
        try:
            r = httpx.get(url, timeout=10)
            r.raise_for_status()
            return r.json()
        except Exception as e:
```

```
            if i == tentativas - 1:
                raise
            time.sleep(2 ** i)  # backoff exponencial
```

Async HTTP

Para múltiplas requisições simultâneas, async é muito mais rápido.

```
import asyncio
import httpx

async def buscar(client: httpx.AsyncClient, url: str) -> dict:
    r = await client.get(url)
    return r.json()

async def buscar_varios(urls: list[str]) -> list[dict]:
    async with httpx.AsyncClient(timeout=30) as client:
        tarefas = [buscar(client, url) for url in urls]
        resultados = await asyncio.gather(*tarefas, return_exceptions=True)
    return resultados

# Executar
urls = [
    "https://api.github.com/users/lorran562",
    "https://api.github.com/repos/lorran562/libertus-core",
]
resultados = asyncio.run(buscar_varios(urls))
```

7. AUTOMAÇÃO E SEGURANÇA

Subprocess — Executar Comandos

```
import subprocess
import shlex

# Executar comando simples
resultado = subprocess.run(
    ["nmap", "-sV", "192.168.1.1"],
    capture_output=True,
    text=True,
    timeout=60
)

print(resultado.stdout)
print(resultado.stderr)
print(resultado.returncode)  # 0 = sucesso

# Shell command (cuidado com injeção!)
# NUNCA faça: subprocess.run(f"nmap {alvo}", shell=True) com input do usuário
# SEMPRE faça:
alvo = "192.168.1.1"
cmd = ["nmap", "-sV", "--script", "vuln", alvo]
resultado = subprocess.run(cmd, capture_output=True, text=True, timeout=120)

# Executar e ler output em tempo real
with subprocess.Popen(
    ["ping", "-c", "4", "8.8.8.8"],
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    text=True
) as proc:
    for linha in proc.stdout:
        print(linha.strip())
```

Regex — Expressões Regulares

```
import re

texto = "IP: 192.168.1.100, porta: 8080, vuln: CVE-2021-44228"

# Encontrar IPs
ips = re.findall(r'\b(?:\d{1,3}\.){3}\d{1,3}\b', texto)
print(ips)  # ['192.168.1.100']

# Encontrar CVEs
cves = re.findall(r'CVE-\d{4}-\d+', texto)
print(cves)  # ['CVE-2021-44228']

# Grupos nomeados
padrao = r'IP: (?P<ip>[\d.]+), porta: (?P<porta>\d+)'
```

```
m = re.search(padrao, texto)
if m:
    print(m.group("ip"))    # 192.168.1.100
    print(m.group("porta")) # 8080

# Substituir
limpo = re.sub(r'\b\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}\b', '[IP]', texto)

# Compilar para reuso (mais rápido)
PATTERN_IP = re.compile(r'\b(?:\d{1,3}\.){3}\d{1,3}\b')
PATTERN_CVE = re.compile(r'CVE-\d{4}-\d+', re.IGNORECASE)

ips = PATTERN_IP.findall(texto)
cves = PATTERN_CVE.findall(texto)
```

Threading e Multiprocessing

```
import threading
import multiprocessing
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

# Thread simples
def escanear_porta(host: str, porta: int, abertas: list):
    import socket
    try:
        s = socket.socket()
        s.settimeout(1)
        s.connect((host, porta))
        s.close()
        abertas.append(porta)
    except:
        pass

# Scan paralelo com ThreadPoolExecutor (I/O bound)
def scan_rapido(host: str, portas: range) -> list:
    abertas = []
    lock = threading.Lock()

    def check(porta):
        import socket
        try:
            s = socket.socket()
            s.settimeout(0.5)
            s.connect((host, porta))
            s.close()
            with lock:
                abertas.append(porta)
        except:
            pass

    with ThreadPoolExecutor(max_workers=100) as executor:
        executor.map(check, portas)

    return sorted(abertas)
```

```python
# ProcessPoolExecutor (CPU bound — usa múltiplos núcleos)
def processar_grande_dataset(dados: list) -> list:
    def processar_chunk(chunk):
        return [item.upper() for item in chunk]

    chunks = [dados[i::4] for i in range(4)]  # divide em 4 partes
    with ProcessPoolExecutor(max_workers=4) as executor:
        resultados = list(executor.map(processar_chunk, chunks))
    return [item for sub in resultados for item in sub]
```

8. BANCO DE DADOS E SUPABASE

Supabase com httpx (padrão LIBERTUS)

```
import httpx
import json
from typing import Any

class SupabaseClient:
    def __init__(self, url: str, key: str):
        self.base = f"{url}/rest/v1"
        self.headers = {
            "apikey": key,
            "Authorization": f"Bearer {key}",
            "Content-Type": "application/json",
            "Prefer": "return=representation"
        }

    def get(self, tabela: str, query: str = "") -> list:
        r = httpx.get(
            f"{self.base}/{tabela}?{query}",
            headers=self.headers,
            timeout=15
        )
        return r.json() if r.status_code == 200 else []

    def post(self, tabela: str, dados: dict) -> dict:
        r = httpx.post(
            f"{self.base}/{tabela}",
            headers=self.headers,
            json=dados,
            timeout=15
        )
        resultado = r.json() if r.text else {}
        return resultado[0] if isinstance(resultado, list) else resultado

    def patch(self, tabela: str, filtro: str, dados: dict):
        httpx.patch(
            f"{self.base}/{tabela}?{filtro}",
            headers=self.headers,
            json=dados,
            timeout=15
        )

    def delete(self, tabela: str, filtro: str):
        httpx.delete(
            f"{self.base}/{tabela}?{filtro}",
            headers=self.headers,
            timeout=15
        )

    def rpc(self, funcao: str, params: dict) -> Any:
        r = httpx.post(
```

```
            f"{self.base.replace('/rest/v1', '')}/rest/v1/rpc/{funcao}",
            headers=self.headers,
            json=params,
            timeout=15
        )
        return r.json() if r.status_code == 200 else None

# Uso
sb = SupabaseClient(
    url="https://jkgrmlfqgprcwfooovkx.supabase.co",
    key="sua_anon_key"
)

# Buscar memórias com peso alto
memorias = sb.get("libertus_memoria", "peso_vivo=gte.7&order=peso_vivo.desc&limit=10")

# Inserir nova memória
sb.post("libertus_memoria", {
    "tipo": "conhecimento",
    "titulo": "Python Completo",
    "conteudo": "...",
    "peso_vivo": 8.0
})

# Atualizar
sb.patch("libertus_memoria", "id=eq.uuid-aqui", {"peso_vivo": 9.0})
```

9. PADRÕES AVANÇADOS

Generators e Iteradores

```
# Generator — produz valores sob demanda (memória eficiente)
def ler_linhas_grandes(arquivo: str):
    with open(arquivo, encoding="utf-8") as f:
        for linha in f:
            yield linha.strip()

# Usar
for linha in ler_linhas_grandes("grande_arquivo.txt"):
    processar(linha)  # nunca carrega tudo na memória

# Generator expression
soma = sum(x**2 for x in range(1_000_000))  # não cria lista

# Generator com send() — bidirecionais
def acumulador():
    total = 0
    while True:
        valor = yield total
        if valor is None:
            break
        total += valor

gen = acumulador()
next(gen)          # inicializa
gen.send(10)       # 10
gen.send(20)       # 30
gen.send(5)        # 35
```

Typing — Type Hints Avançados

```
from typing import Union, Optional, Any, Callable, TypeVar, Generic
from typing import Protocol

# Union e Optional
def processar(valor: Union[str, int, None]) -> Optional[str]:
    if valor is None:
        return None
    return str(valor)

# TypeVar — generics
T = TypeVar('T')

def primeiro(lista: list[T]) -> Optional[T]:
    return lista[0] if lista else None

# Protocol — duck typing formal
class Processavel(Protocol):
    def processar(self, entrada: str) -> str: ...
```

```
    def esta_disponivel(self) -> bool: ...

def executar(agente: Processavel, entrada: str) -> str:
    if not agente.esta_disponivel():
        return "indisponível"
    return agente.processar(entrada)

# Callable
Handler = Callable[[str, dict], str]

def registrar_handler(evento: str, fn: Handler) -> None:
    print(f"Handler registrado para {evento}")
```

10. BOAS PRÁTICAS E PERFORMANCE

Boas Práticas

```
# 1. Use f-strings, não format() ou %
nome = "LIBERTUS"
msg = f"Sistema: {nome}"  # correto
msg = "Sistema: {}".format(nome)  # evite
msg = "Sistema: %s" % nome  # evite

# 2. Use list/dict/set comprehensions
quadrados = [x**2 for x in range(10)]  # correto
quadrados = []
for x in range(10):                    # verboso
    quadrados.append(x**2)

# 3. Use enumerate() ao invés de range(len())
for i, item in enumerate(lista):       # correto
    pass
for i in range(len(lista)):            # evite
    item = lista[i]

# 4. Use zip() para iterar múltiplas listas
for a, b in zip(lista1, lista2):       # correto
    pass

# 5. Use any() e all()
tem_admin = any(u["role"] == "admin" for u in usuarios)
todos_ativos = all(u["ativo"] for u in usuarios)

# 6. Walrus operator := (Python 3.8+)
if (n := len(lista)) > 10:
    print(f"Lista grande: {n} itens")

# 7. Use dataclasses para structs
from dataclasses import dataclass

@dataclass
class Resultado:
    sucesso: bool
    dados: dict
    erro: str = ""

# 8. Nunca use mutable default args
def errado(lista=[]):  # BUG: lista é compartilhada entre chamadas!
    lista.append(1)
    return lista

def correto(lista=None):
    if lista is None:
        lista = []
    lista.append(1)
    return lista
```

Performance

```
import time
from functools import lru_cache

# 1. lru_cache — memoização automática
@lru_cache(maxsize=128)
def fibonacci(n: int) -> int:
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

# 2. Use sets para busca O(1) ao invés de list O(n)
lista = [1, 2, 3, 4, 5]
conjunto = {1, 2, 3, 4, 5}

5 in lista      # O(n) — percorre tudo
5 in conjunto   # O(1) — hash lookup

# 3. Join ao invés de concatenação
partes = ["a", "b", "c", "d"]
resultado = "".join(partes)  # correto — O(n)
resultado = ""
for p in partes:             # errado — O(n²)
    resultado += p

# 4. Slots para classes com muitas instâncias
class Ponto:
    __slots__ = ['x', 'y']  # economiza memória, acesso mais rápido

    def __init__(self, x, y):
        self.x = x
        self.y = y

# 5. Profiling
import cProfile
import pstats

with cProfile.Profile() as pr:
    resultado = funcao_lenta()

stats = pstats.Stats(pr)
stats.sort_stats("cumulative")
stats.print_stats(10)  # top 10 funções mais lentas
```